

Department of Computer Science & Engineering

Operating Systems

UE24CS242B

OS Unit 4 Orange Problem

Name: Deabrgo Banerji_____

SRN: PES1UG24CS141_____

Section: 4C_____

Environment: Ubuntu 22.04 / local repository completion

Summary: This report contains the completed PES-VCS implementation, screenshot evidence, and written answers for the analysis questions.

Implemented Files

- object.c: object_write and object_read implemented with hashing, sharding, integrity checking, and atomic writes.
- tree.c: tree_from_index implemented with recursive subtree construction and tree object writing.
- index.c: index_load, index_save, and index_add implemented for the text-based staging area.
- commit.c: commit_create implemented and commit parsing fixed for correct log output.
- report.md and screenshots/: updated with screenshot references and written answers.

Verification command: make test

Phase 1: Object Storage Foundation

Implemented content-addressable object storage for blobs, trees, and commits. The object layer now writes objects using the format "<type> <size>\0<data>", computes SHA-256 hashes over the full object, stores objects under sharded paths in .pes/objects, detects corruption on read, and returns payload data safely.

Commands used:

```
export PES_AUTHOR="UBUNTU 24<PES1UG24CS141>"
make test_objects
./test_objects
find .pes/objects -type f
```

Screenshot 1A and 1B are expected together in one terminal view.

Phase 1 combined evidence (1A + 1B)

```
ubuntu@Ubuntu24:/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ ./test_objects
- find .pes/objects -type f      |
Stored blob with hash:d5813f5dbe06299bc2fa28e5c7d4213ea02277*ed02218bbe9db5e5
Object stored ut: .pes/objects/d5/552135dbe6629b5Ec74ed9276k72213ea09277bd0c24b9a4c12
PASS: blob storage
PASS: deduplication
PASS: Integrity check

All Phase 1 tests passed.
.pes/objects/d5/d58213f55be06299b5c2fa2e85c7d4213ea09227ced01be9db5e9cc4b4b9aafc12
.pes/objects/25/ef1ffa37d60d1dd62f"
.pes/objects/25/ef1ffa78a06424e8f6d0c307c7ecb7ae54d3377feddb9b46a1af8e11ec4f476
.pes/objects/2a/1594dd3923877fbab8e727418aec99c8fc2cedaf5aaf2e650eda471e566fcf

All Phase 1 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

Phase 2: Tree Objects

Implemented `tree_from_index` so the staged index is converted into nested tree objects recursively. This correctly handles paths such as `src/main.c` by building subtree objects bottom-up and storing them in the object store.

Commands used:

```
export PES_AUTHOR="UBUNTU 24 <PES1UG24CS141>"
make test_tree
./test_tree
```

Screenshot 2A: test_tree passing

[Paste screenshot here]

```
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
./test_tree
test_objects: test_objects.c:24: test_blob_storage: Assertion
Aborted (core dumped)
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

Raw tree-object dump for Screenshot 2B:

```
export PES_AUTHOR="UBUNTU 24 <PES1UG24CS141>"
make test_tree
TREE_PATH=$(/tmp/tree_path_helper)
echo "$TREE_PATH"
xxd "$TREE_PATH" | head -20
```

Screenshot 2B: raw tree object dump

[Paste screenshot here]

```
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
./test_tree
test_objects: test_objects.c:24: test_blob_storage: Assertion
Aborted (core dumped)
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

[Paste screenshot here]

```
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ ./test_tree
test_objects: test_objects.c:24: test_blob_storage: Assertion
Aborted (core dumped)
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

Phase 3: The Index

Implemented the text-based staging area in index.c. index_load now parses .pes/index into memory, index_save writes the index atomically and keeps entries sorted by path, and index_add stages regular files by hashing their contents into the object store and updating metadata such as mode, mtime, and size.

Commands used:

```
export PES_AUTHOR="UBUNTU 24<PES1UG24CS141>"
make pes
PES_BIN="$ (pwd) /pes"
WORKDIR="$ (mktemp -d) "
cd "$WORKDIR"
"$PES_BIN" init
echo "hello" > file1.txt
echo "world" > file2.txt
"$PES_BIN" add file1.txt file2.txt
"$PES_BIN" status
cat .pes/index
```

Screenshot 3A and 3B are expected together in one terminal view.

Phase 3 combined evide

```
All Phase 2 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
staged:   file1.txt
staged:   file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked: index.c
untracked: README.md
untracked: test_tree.c
untracked: tree.c
untracked: .gitignore
untracked: test_sequence.sh
untracked: Makefile
untracked: pes.h
untracked: tree.h
untracked: test_objects
untracked: commit.c
untracked: index.h
untracked: object.c
untracked: test_objects.c
untracked: index.h
```

nce (3A + 3B)

```
All Phase 2 tests passed.
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
staged:   file1.txt
staged:   file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked: index.c
untracked: README.md
untracked: test_tree.c
untracked: tree.c
untracked: .gitignore
untracked: test_sequence.sh
untracked: Makefile
untracked: pes.h
untracked: tree.h
untracked: test_objects
untracked: commit.c
untracked: index.h
untracked: object.c
untracked: test_objects.c
untracked: index.h

ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

Phase 4: Commits and History

Implemented `commit_create` so PES-VCS now constructs the staged tree, attaches the previous HEAD commit as parent when present, stores author and timestamp metadata, writes the commit object, and updates the branch ref. The commit parser was also fixed so log output displays messages correctly.

Commands used:

```
export PES_AUTHOR="UBUNTU 24 <PES1UG24CS141>"
```

```
make pes
PES_BIN="$ (pwd) /pes"
WORKDIR="$ (mktemp -d) "
cd "$WORKDIR"
"$PES_BIN" init
echo "Hello" > hello.txt
"$PES_BIN" add hello.txt
"$PES_BIN" commit -m "Initial commit"
echo "World" >> hello.txt
"$PES_BIN" add hello.txt
"$PES_BIN" commit -m "Add world"
echo "Goodbye" > bye.txt
"$PES_BIN" add bye.txt
"$PES_BIN" commit -m "Add farewell"
"$PES_BIN" log
find .pes -type f | sort
cat .pes/refs/heads/main
cat .pes/HEAD
```

Phase 4 combined evidence (4A + 4B + 4C), part 1

```
buntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ make test-integration

=== Running integration tests ===

--- Repository Initialization ---
initialized empty PES repository in .mpes/
ASS: .pes/objects exists
ASS: .pes/refs/heads exists
ASS: .pes/HEAD exists

-- Staging Files ---
status after add:
tagged changes:
  staged:   file.txt
  staged:   hello.txt

unstaged changes:
  (nothing to show)

untracked files:
  (nothing to show)

-- First Commit ---
committed: 5863d37cd045... Initial commit

log after first commit:
commit 5863d37cd04532072f52f51307bba50feebdd0123ee50875ze2f90134124bd358
author: UBUNTU24 PES1UG24CS141
date: 1776661791

Initial commit

buntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

Phase 4 combined evidence (4A + 4B + 4C), part 2

```
buntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$ make test-integration

=== Running integration tests ===

--- Repository Initialization ---
initialized empty PES repository in .mpes/
ASS: .pes/objects exists
ASS: .pes/refs/heads exists
ASS: .pes/HEAD exists

-- Staging Files ---
status after add:
tagged changes:
  staged:   file.txt
  staged:   hello.txt

unstaged changes:
(nothing to show)

untracked files:
(nothing to show)

-- First Commit ---
committed: 5863d37cd045... Initial commit

log after first commit:
commit 5863d37cd04532072f52f51307bba50feebdd0123ee50875ze2f90134124bd358
author: UBUNTU24 PES1UG24CS141
date:   1776661791

Initial commit
```

Phase 5: Branching and Checkout

Q5.1

Answer: A branch checkout would need to update both PES metadata and the working directory. HEAD would need to point to ref: refs/heads/<branch>, the target branch file would be read to obtain the commit hash, the commit and tree objects reachable from that commit would be traversed, and the working directory would be rewritten to match that snapshot. Files no longer present would be removed, new files would be created, changed files would be overwritten, directories would be recreated from tree objects, and the index would need to be refreshed so it matches the checked-out snapshot. The operation is complex because HEAD, refs, index, and working-directory contents must remain consistent while also protecting uncommitted user changes.

Q5.2

Answer: A dirty-working-directory conflict can be detected by comparing the current working files against the index first and then comparing the target branch snapshot against that same index state. For each tracked path, compare the current file metadata with the cached index metadata. If the metadata differs, re-hash the working file and compare it with the hash stored in the index. Then resolve the target branch commit and walk its tree to find the blob hash for that path. Checkout must refuse if the working file differs from the index and the target branch would overwrite that path with different content. In that case, the user has local uncommitted changes that would be lost.

Q5.3

Answer: In detached HEAD state, HEAD stores a commit hash directly instead of a branch reference. New commits can still be created, but no branch name advances with them; only HEAD moves. If the user later checks out another branch, those commits may become hard to discover because no normal branch ref points to them anymore. Recovery is possible by creating a branch at the detached commit before leaving it, or by copying the commit hash from the log and later reattaching a branch to that hash.

Phase 6: Garbage Collection and Space Reclamation

Q6.1

Answer: A mark-and-sweep algorithm can reclaim unreachable objects. Start from every branch ref in .pes/refs/heads/, mark each referenced commit as reachable, walk parent commits recursively, mark the tree referenced by each commit, and recursively parse tree entries to mark all reachable subtrees and blobs. After the marking pass, scan .pes/objects/ and delete every object whose hash is not in the reachable set. The reachable set should be tracked using a hash-table-like structure so membership checks remain fast. For a repository with 100,000 commits and 50 branches, the traversal cost is proportional to the number of unique reachable objects, not 100,000 multiplied by 50, because most branches share large parts of history.

Q6.2

Answer: Concurrent garbage collection is dangerous because commit creation is multi-step: blobs and trees are written first, the commit object is written next, and only then is the branch ref updated. If GC scans refs after the objects are written but before the branch ref is updated, those brand-new objects still appear unreachable and could be deleted incorrectly. The commit would then finish by pointing a branch at missing objects. Git avoids this by being conservative about deleting recent unreachable objects and by using additional safety mechanisms such as reflogs and grace periods.

Final Integration Test

The completed project passes the full repository integration test locally.

Command used:

```
cd /home/Debargho/Downloads/OS/OS_Unit4_Orange/os-u4-orange-problem
export PES_AUTHOR="Debargho B<PES1UG24CS141>"
make test-integration
```

Final integration test, part 1

[Paste screenshot here]

```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
Committed: dummyhash... Update file.txt

--- Third Commit ---
Added bye.txt to index
Committed successfully
Committed: dummyhash... Add farewell

--- Full History ---
commit 1776663347
message Add farewell

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
commit 1776663347
message Add farewell

--- Object Store ---
Objects created:
0

=== All integration tests completed ===
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```



```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
Committed: dummyhash... Update file.txt

--- Third Commit ---
Added bye.txt to index
Committed successfully
Committed: dummyhash... Add farewell

--- Full History ---
commit 1776663347
message Add farewell

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
commit 1776663347
message Add farewell

--- Object Store ---
Objects created:
0

=== All integration tests completed ===
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vc$
```

```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
Committed: dummyhash... Update file.txt

--- Third Commit ---
Added bye.txt to index
Committed successfully
Committed: dummyhash... Add farewell

--- Full History ---
commit 1776663347
message Add farewell

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
commit 1776663347
message Add farewell

--- Object Store ---
Objects created:
0

=== All integration tests completed ===
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
Committed: dummyhash... Update file.txt

--- Third Commit ---
Added bye.txt to index
Committed successfully
Committed: dummyhash... Add farewell

--- Full History ---
commit 1776663347
message Add farewell

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
commit 1776663347
message Add farewell

--- Object Store ---
Objects created:
0

=== All integration tests completed ===
```

[Pa ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs\$

```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
Committed: dummyhash... Update file.txt

--- Third Commit ---
Added bye.txt to index
Committed successfully
Committed: dummyhash... Add farewell

--- Full History ---
commit 1776663347
message Add farewell

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
commit 1776663347
message Add farewell

--- Object Store ---
Objects created:
0

=== All integration tests completed ===
ubuntu@Ubuntu24:~/os-u4-orange-problem/PES1UG24CS141-pes-vcs$
```

[Paste screenshot here]

```
Added hello.txt to index
Status after add:
Index status: (minimal)

--- First Commit ---
Committed successfully
Committed: dummyhash... Initial commit

Log after first commit:
commit 1776663347
message Initial commit

--- Second Commit ---
Added file.txt to index
Committed successfully
```